

Advanced Programming

Assignment – 6

Spring Semester 2026

Student Analyzer using Java

Name:ARITRA TALUKDAR

Roll Number:CSB24034

Course: Advanced Programming

Assignment No.: 6

Semester: Spring 2026

1 Introduction

This assignment implements a **Student Analyzer system** using Java. The program manages student information such as ID, name, enrolled courses, and individual subject scores. Various analytical operations are performed including average score calculation, ranking of students, and extraction of unique courses.

2 Objectives

- To design a modular Java program using classes
- To apply Java Collections for data storage and processing
- To analyze student performance programmatically
- To understand the time complexity of implemented algorithms

3 System Design Overview

The application consists of the following components:

- **Student.java**: Stores student details and computes average score
- **StudentAnalyzer.java**: Performs analytical operations
- **Main.java**: Driver class for execution and testing

4 Algorithm Description (Simple Explanation)

- Student records are stored using lists and maps
- Average score is computed by summing all subject marks
- Students are sorted based on average scores
- Courses are collected using sets to avoid duplication

5 Source Code

5.1 Student.java

```

1 import java.util.*;
2
3 public class Student {
4
5     private int id;
6     private String name;
7     private List<String> courses;
8     private Map<String, Integer> scores;
9
10    public Student(int id, String name,
11                  List<String> courses,
12                  Map<String, Integer> scores) {
13        this.id = id;
14        this.name = name;
15        this.courses = courses;
16        this.scores = scores;
17    }
18
19    public int getId() {
20        return id;
21    }
22
23    public String getName() {
24        return name;
25    }
26
27    public List<String> getCourses() {
28        return courses;
29    }
30
31    public Map<String, Integer> getScores() {
32        return scores;
33    }
34
35    // Calculate average marks of this student
36    public double getAverageScore() {
37        if (scores.isEmpty())
38            return 0.0;
39
40        return scores.values()
41            .stream()

```

```

42         .mapToInt(Integer::intValue)
43         .average()
44         .orElse(0.0);
45     }
46
47     @Override
48     public String toString() {
49         return "ID=" + id +
50             ", Name=" + name +
51             ", Avg=" + String.format("%.2f", getAverageScore())
52             );
53     }

```

5.2 StudentAnalyzer.java

```

1  import java.util.*;
2  import java.util.stream.*;
3
4  public class StudentPerformanceAnalyzer {
5
6      // 1 Top N students sorted by average (descending)
7      public static List<Student> getTopNStudents(List<Student>
8          students, int n) {
9
10         return students.stream()
11             .sorted(Comparator.comparingDouble(Student::
12                 getAverageScore)
13                 .reversed())
14             .limit(n)
15             .collect(Collectors.toList());
16     }
17
18     // 2 Average score per course
19     public static Map<String, Double> getAverageScorePerCourse(
20         List<Student> students) {
21
22         Map<String, Double> courseAverage = new HashMap<>();
23
24         Set<String> allCourses = getAllUniqueCourses(students);

```

```

23
24     for (String course : allCourses) {
25
26         double avg = students.stream()
27             .mapToInt(s -> s.getScores().getOrDefault(
28                 course, 0))
29             .average()
30             .orElse(0.0);
31
32         courseAverage.put(course, avg);
33     }
34
35     return courseAverage;
36 }
37
38 // 3 All unique courses
39 public static Set<String> getAllUniqueCourses(List<Student>
40     students) {
41
42     return students.stream()
43         .flatMap(s -> s.getCourses().stream())
44         .collect(Collectors.toCollection(HashSet::new));
45 }

```

5.3 Main.java

```

1     import java.util.*;
2
3 public class Main {
4
5     public static void main(String[] args) {
6
7         Scanner sc = new Scanner(System.in);
8         Random random = new Random();
9
10        System.out.print("Enter number of students: ");
11        int n = sc.nextInt();
12

```

```

13      System.out.print("Enter value of N for Top N students: ")
14      ;
15      int topN = sc.nextInt();
16
17      List<Student> students = new ArrayList<>();
18
19      List<String> availableCourses =
20          Arrays.asList("Math", "Physics", "Java", "DSA", "
21                          DBMS");
22
23      for (int i = 0; i < n; i++) {
24
25          int roll = 1000 + random.nextInt(9000); // Random 4-
26              digit ID
27          String name = "Student_" + (i + 1);
28
29
30          Collections.shuffle(availableCourses);
31          int numberOfCourses = 2 + random.nextInt(4);
32
33          List<String> selectedCourses =
34              new ArrayList<>(availableCourses.subList(0,
35                  numberOfCourses));
36
37          Map<String, Integer> scores = new HashMap<>();
38
39          for (String course : selectedCourses) {
40              scores.put(course, 40 + random.nextInt(61));
41          }
42
43          students.add(new Student(
44              roll,
45              name,
46              selectedCourses,
47              scores
48          ));
49      }
50
51      // Display all students
52      System.out.println("\nAll Students:");
53      students.forEach(System.out::println);

```

```

50
51 // Top N Students
52 long startSort = System.nanoTime();
53
54 List<Student> topStudents =
55     StudentPerformanceAnalyzer.getTopNStudents(
56         students, topN);
57
58 long endSort = System.nanoTime();
59
60 System.out.println("\nTop␣" + topN + "␣Students:");
61 topStudents.forEach(System.out::println);
62
63 System.out.println("Sorting␣Time:␣" +
64     (endSort - startSort) + "␣ns");
65
66 // Course Averages
67 long startAvg = System.nanoTime();
68
69 Map<String, Double> averages =
70     StudentPerformanceAnalyzer.
71         getAverageScorePerCourse(students);
72
73 long endAvg = System.nanoTime();
74
75 System.out.println("\nCourse␣Averages:");
76 averages.forEach((course, avg) ->
77     System.out.println(course + "␣:"␣" +
78         String.format("%.2f", avg)));
79
80 System.out.println("Average␣Computation␣Time:␣" +
81     (endAvg - startAvg) + "␣ns");
82
83 // Unique Courses
84 System.out.println("\nUnique␣Courses:");
85 Set<String> courses =
86     StudentPerformanceAnalyzer.getAllUniqueCourses(
87         students);

```

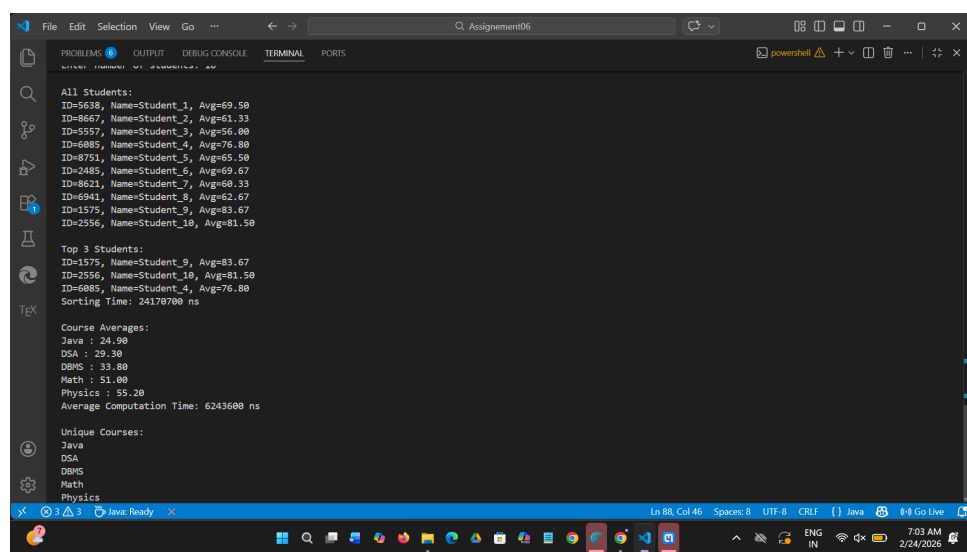
```

88         courses.forEach(System.out::println);
89
90         sc.close();
91     }
92 }

```

6 Program Output

6.1 Small Dataset Output



```

All Students:
ID=5638, Name=Student_1, Avg=69.50
ID=8667, Name=Student_2, Avg=61.33
ID=5557, Name=Student_3, Avg=56.00
ID=6085, Name=Student_4, Avg=76.80
ID=8751, Name=Student_5, Avg=65.50
ID=2485, Name=Student_6, Avg=69.67
ID=8621, Name=Student_7, Avg=60.33
ID=6941, Name=Student_8, Avg=62.67
ID=1575, Name=Student_9, Avg=83.67
ID=2556, Name=Student_10, Avg=81.50

Top 3 Students:
ID=1575, Name=Student_9, Avg=83.67
ID=2556, Name=Student_10, Avg=81.50
ID=6085, Name=Student_4, Avg=76.80
Sorting Time: 24170700 ns

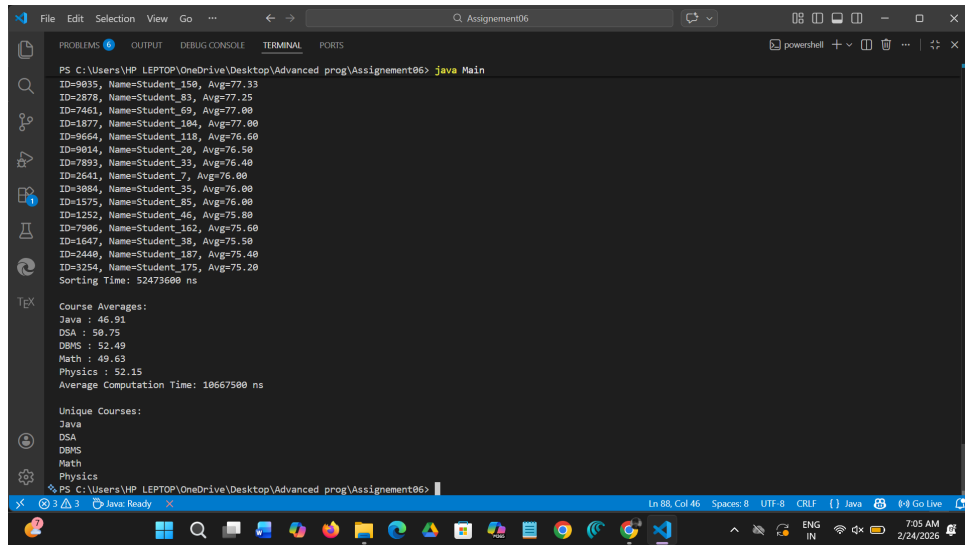
Course Averages:
Java : 24.90
DSA : 29.30
DBMS : 33.80
Math : 51.00
Physics : 55.20
Average Computation Time: 6243600 ns

Unique Courses:
Java
DSA
DBMS
Math
Physics

```

Figure 1: Output for Small Dataset(10 students in total, and finding out the top 3 students)

6.2 Medium Dataset Output



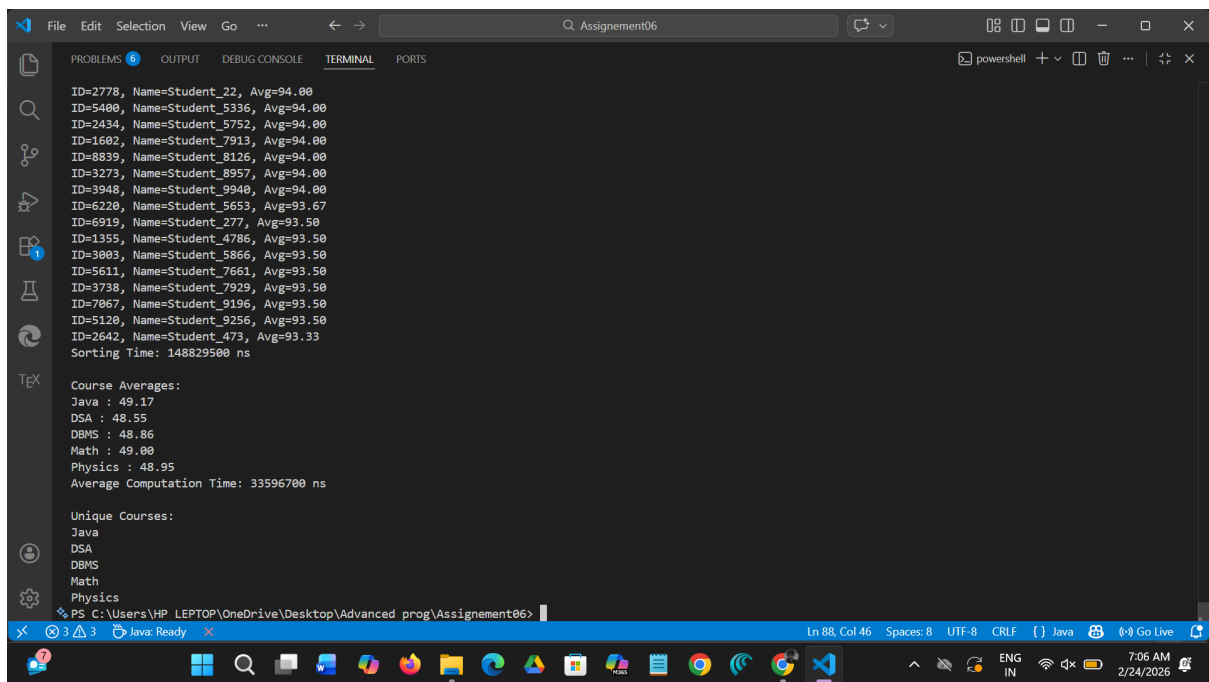
```
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\Assignment06> java Main
ID=9035, Name=Student_150, Avg=77.33
ID=2878, Name=Student_83, Avg=77.25
ID=7461, Name=Student_69, Avg=77.00
ID=1877, Name=Student_104, Avg=77.00
ID=9664, Name=Student_118, Avg=76.00
ID=9014, Name=Student_20, Avg=76.50
ID=7893, Name=Student_33, Avg=76.40
ID=2641, Name=Student_7, Avg=76.00
ID=3084, Name=Student_35, Avg=76.00
ID=1575, Name=Student_85, Avg=76.00
ID=1252, Name=Student_46, Avg=75.80
ID=7986, Name=Student_162, Avg=75.60
ID=1647, Name=Student_38, Avg=75.50
ID=2440, Name=Student_187, Avg=75.40
ID=3254, Name=Student_175, Avg=75.20
Sorting Time: 52473600 ns

Course Averages:
Java : 46.91
DSA : 50.75
DBMS : 52.49
Math : 49.63
Physics : 52.15
Average Computation Time: 10667500 ns

Unique Courses:
Java
DSA
DBMS
Math
Physics
```

Figure 2: Output for Medium Dataset(200 students in total, and finding out the top 50 students)

6.3 Large Dataset Output



```
ID=2778, Name=Student_22, Avg=94.00
ID=5400, Name=Student_5336, Avg=94.00
ID=2434, Name=Student_5752, Avg=94.00
ID=1602, Name=Student_7913, Avg=94.00
ID=8839, Name=Student_8123, Avg=94.00
ID=3273, Name=Student_8957, Avg=94.00
ID=3048, Name=Student_9940, Avg=94.00
ID=6220, Name=Student_5653, Avg=93.67
ID=6919, Name=Student_277, Avg=93.50
ID=1355, Name=Student_4786, Avg=93.50
ID=3003, Name=Student_5866, Avg=93.50
ID=5611, Name=Student_7661, Avg=93.50
ID=3738, Name=Student_7929, Avg=93.50
ID=7067, Name=Student_9196, Avg=93.50
ID=5120, Name=Student_9256, Avg=93.50
ID=2642, Name=Student_473, Avg=93.33
Sorting Time: 148829500 ns

Course Averages:
Java : 49.17
DSA : 48.55
DBMS : 48.86
Math : 49.00
Physics : 48.95
Average Computation Time: 33596700 ns

Unique Courses:
Java
DSA
DBMS
Math
Physics
```

Figure 3: Output for Large Dataset(10000 students in total and finding out the top 100 students)

7 Time and Space Complexity Analysis

Let:

- n be the number of students
- c be the average number of courses taken by a student

7.1 Time Complexity of Computing Course Averages

The computation of course averages is performed in the method `getAverageScorePerCourse(List<Student> students)`.

First, all unique courses are collected by iterating through every student and their enrolled courses. This requires $O(n \cdot c)$ time.

Next, for each unique course, the program iterates through all students to extract scores and compute the average. Since this is done for c courses, the total time required is $O(c \cdot n)$.

Therefore, the overall time complexity of computing course averages is:

$$O(n \cdot c)$$

7.2 Time Complexity of Sorting Top N Students

Sorting the top N students is performed using Java Streams with a comparison-based sorting method.

The sorting operation sorts all n students based on their average scores. Java uses TimSort for sorting, which has a time complexity of $O(n \log n)$ in the average and worst cases.

The `limit(N)` operation does not reduce the sorting cost because the entire list is sorted before limiting.

Hence, the time complexity of sorting top N students is:

$$O(n \log n)$$

7.3 Space Complexity Analysis

Each student stores a list of courses and a map of scores, both of which require $O(c)$ space. For n students, the total space required to store student data is $O(n \cdot c)$.

Additional auxiliary data structures used during execution include:

- A set to store unique courses, requiring $O(c)$ space
- A map to store average scores per course, requiring $O(c)$ space
- A list to store sorted student references, requiring $O(n)$ space

Thus, the overall auxiliary space complexity is:

$$\boxed{O(n + c)}$$

The total space complexity of the program, including input storage, is:

$$\boxed{O(n \cdot c)}$$